Copy Table

목적

Copy Table은 **메모리, 바이트코드, 트랜잭션 데이터 간의 복사 연산을 추적하고 검증**하는 테이블입니다. 이 테이블은 Copy Circuit에서 생성되며, EVM Circuit에서 CALLDATACOPY, CODECOPY, EXTCODECOPY, RETURNDATACOPY, MCOPY, LOG 등의 opcode 실행 시 사용됩니다. 각 복사 이벤트는 소스와 목적지 간의 바이트 단위 복사를 추적합니다.

테이블 스키마

파일: `zkevm-circuits/src/table.rs` (line 1728)

```
pub struct CopyTable {
/// 행 활성화 여부
pub q_enable: Column<Fixed>,
/// 복사 이벤트의 첫 번째 행 여부
pub is_first: Column<Advice>,
/// ID (Call ID, Bytecode Hash, Tx ID 등)
pub id: Column<Advice>,
/// 주소 (메모리 주소, 바이트코드 인덱스 등)
pub addr: Column<Advice>,
/// 소스 버퍼의 끝 주소
pub src_addr_end: Column<Advice>,
```

```rust
/// 남은 실제 바이트 수
pub real_bytes_left: Column<Advice>,
/// 값 word RLC (사용 예정)
pub value_wrod_rlc: Column<Advice>,
/// RLC 누적 값
pub rlc_acc: Column<Advice>,
/// Read-Write 카운터
pub rw_counter: Column<Advice>,
/// 남은 RW 카운터 증가량
pub rwc_inc_left: Column<Advice>,
/// 복사 데이터 타입 태그
pub tag: BinaryNumberConfig<CopyDataType, { CopyDataType::N_BITS }>,
}
```

컬럼 설명

1. q_enable (Fixed)

- 해당 행이 유효한 복사 행인지 표시

2. is_first (Advice)

- 복사 이벤트의 첫 번째 read-write 쌍인지 표시
- `1` : 첫 번째 행
- `0` : 이후 행

3. id (Advice, SecondPhase)

- 복사 소스/목적지의 ID (RLC 인코딩)
- **Memory**: Call ID
- **Bytecode**: Bytecode Hash (RLC)
- **TxCalldata/TxLog**: Transaction ID

4. addr (Advice)

- 복사 소스/목적지의 주소
- **Memory**: 메모리 주소 (바이트 단위)
- **Bytecode**: 바이트 인덱스
- **TxCalldata**: CallData 인덱스
- **TxLog**: 로그 주소 (tx_id, log_id, field_tag, index 조합)

5. src_addr_end (Advice)

- 소스 버퍼의 끝 주소
- 이 주소 이상에서 읽으면 0 반환 (패딩)

6. real_bytes_left (Advice)

- 마스크되지 않은 남은 바이트 수
- 복사 진행에 따라 감소

7. value_wrod_rlc (Advice)

- 값 word RLC (향후 사용 예정)

8. rlc_acc (Advice, SecondPhase)

- RLC 누적 값
- 특정 복사 타입(예: RlcAcc)에서 사용

- Challenge를 사용하여 바이트 시퀀스를 하나의 필드 원소로 압축

9. rw_counter (Advice)

- 현재 read-write 연산의 RW 카운터

10. rwc_inc_left (Advice)

- 남은 RW 카운터 증가량
- 복사 진행에 따라 감소

11. tag (BinaryNumberConfig)

- 복사 데이터 타입을 식별하는 태그
- Binary number chip을 사용하여 효율적으로 인코딩

CopyDataType 열거형

파일: `bus-mapping/src/circuit_input_builder.rs`

```
pub  enum CopyDataType {
Memory, // 메모리
Bytecode, // 바이트코드
TxCalldata, // 트랜잭션 CallData
TxLog, // 트랜잭션 로그
RlcAcc, // RLC 누적 (특수 타입)
Padding, // 패딩 (내부 타입)
AccessListAddresses, // Access List 주소
AccessListStorageKeys, // Access List 스토리지 키
```

```
// 프리컴파일 타입들
Sha256,
Modexp,
EcAdd,
EcMul,
EcPairing,
}
```

데이터 타입 설명

1. Memory

- EVM 메모리 간 복사
- **Opcode**: CALLDATACOPY, CODECOPY, RETURNDATACOPY, MCOPY, LOG 등
- **ID**: Call ID
- **주소**: 메모리 주소 (바이트 단위)

2. Bytecode

- 바이트코드에서 메모리로 복사
- **Opcode**: CODECOPY, EXTCODECOPY
- **ID**: Bytecode Hash (RLC)
- **주소**: 바이트 인덱스

3. TxCalldata

- 트랜잭션 CallData에서 메모리로 복사
- **Opcode**: CALLDATACOPY

- **ID**: Transaction ID
- **주소**: CallData 인덱스

4. TxLog

- 메모리에서 트랜잭션 로그로 복사
- **Opcode**: LOG0, LOG1, LOG2, LOG3, LOG4
- **ID**: Transaction ID
- **주소**: 로그 주소 (조합)

5. RlcAcc

- RLC 누적 값 계산용 특수 타입
- 바이트 시퀀스를 RLC로 인코딩

6. Padding

- 내부 타입 (소스 버퍼 끝 이후 0 패딩)

7. AccessListAddresses / AccessListStorageKeys

- EIP-2930 Access List 복사
- **ID**: Transaction ID

8. 프리컴파일 타입들

- SHA256, MODEXP, EcAdd, EcMul, EcPairing
- 프리컴파일 입력/출력 데이터 복사

복사 이벤트 구조

각 복사 이벤트는 **read-write 쌍**으로 구성됩니다.

Read-Write 쌍

하나의 바이트 복사는 2개의 행으로 표현됩니다:

1. **Read 행**: 소스에서 바이트 읽기
2. **Write 행**: 목적지에 바이트 쓰기

예시: 메모리 간 1바이트 복사

```
is_first | tag | id | addr | src_addr_end | real_bytes_left | rw_counter

---------|--------|----| -----|--------------|-----------------|------------

1 | Memory | 1 | 100 | 101 | 1 | 500 (Read)

0 | Memory | 1 | 200 | 201 | 1 | 501 (Write)
```

복사 이벤트 예시

CALLDATACOPY(dest=0, offset=0, length=4)

CallData: `0x60806040`

Copy Table:

```
is_first | tag | id | addr | src_addr_end | real_bytes_left | rw_counter

---------|------------|----| -----|--------------|-----------------|------------

1 | TxCalldata | 1 | 0 | 4 | 4 | 100 (Read)

0 | Memory | 1 | 0 | 4 | 4 | 101 (Write)

0 | TxCalldata | 1 | 1 | 4 | 3 | 102 (Read)

0 | Memory | 1 | 1 | 4 | 3 | 103 (Write)

0 | TxCalldata | 1 | 2 | 4 | 2 | 104 (Read)
```

```
0 | Memory | 1 | 2 | 4 | 2 | 105 (Write)

0 | TxCalldata | 1 | 3 | 4 | 1 | 106 (Read)

0 | Memory | 1 | 3 | 4 | 1 | 107 (Write)
```

소스 버퍼 끝 처리

소스 주소가 `src_addr_end` 이상이면 **0을 반환**합니다 (패딩).

예시: CALLDATACOPY(dest=0, offset=0, length=10), CallData 길이=4

```
addr | src_addr_end | is_pad | value

-----|--------------|--------|-------

0 | 4 | 0 | 0x60

1 | 4 | 0 | 0x80

2 | 4 | 0 | 0x60

3 | 4 | 0 | 0x40

4 | 4 | 1 | 0x00 (패딩)

5 | 4 | 1 | 0x00 (패딩)

6 | 4 | 1 | 0x00 (패딩)

7 | 4 | 1 | 0x00 (패딩)

8 | 4 | 1 | 0x00 (패딩)

9 | 4 | 1 | 0x00 (패딩)
```

RLC 누적 (rlc_acc)

일부 복사 이벤트는 **RLC 누적 값**을 계산합니다.

계산 방식

```
// 마스크되지 않은 바이트만 누적
if !mask {
rlc_acc = rlc_acc * challenge + value
}
```

사용 사례

- **CALLDATACOPY**: CallData RLC 검증
- **RETURNDATACOPY**: Return data RLC 검증
- **LOG**: 로그 데이터 RLC 검증

예시: CallData RLC 누적

```
addr | value | mask | rlc_acc
-----|-------|------|------------------
0 | 0x60 | 0 | 0x60
1 | 0x80 | 0 | 0x60 * r + 0x80
2 | 0x60 | 0 | (0x60 * r + 0x80) * r + 0x60
3 | 0x40 | 0 | ((0x60 * r + 0x80) * r + 0x60) * r + 0x40
```

MCOPY (EIP-5656) 특수 처리

`MCOPY` 는 메모리 내 복사로, 소스와 목적지가 겹칠 수 있습니다.

처리 방식

1. **읽기 단계**: 모든 소스 바이트를 먼저 읽음
2. **쓰기 단계**: 읽은 바이트를 목적지에 씀

예시: MCOPY(dest=2, src=0, length=4)

```
단계 | tag | addr | 설명
-----|--------|------|------------------
Read | Memory | 0 | 소스[0] 읽기
Read | Memory | 1 | 소스[1] 읽기
Read | Memory | 2 | 소스[2] 읽기
Read | Memory | 3 | 소스[3] 읽기
Write| Memory | 2 | 목적지[2] 쓰기
Write| Memory | 3 | 목적지[3] 쓰기
Write| Memory | 4 | 목적지[4] 쓰기
Write| Memory | 5 | 목적지[5] 쓰기
```

이 방식으로 겹치는 영역도 올바르게 처리됩니다.

EVM Circuit과의 연동

EVM Circuit은 Copy Table을 조회하여 복사 연산을 검증합니다.

Opcode별 사용

CALLDATACOPY

```
// CALLDATACOPY(dest_offset, offset, length)
let  dest_offset = stack.pop();
let  offset = stack.pop();
let  length = stack.pop();


// Copy Table lookup
for  i  in  0..length {
// Read from TxCalldata
lookup_copy_table(
is_first: i == 0,
tag: CopyDataType::TxCalldata,
id: tx_id,
addr: offset + i,
src_addr_end: calldata_length,
...
);
// Write to Memory
lookup_copy_table(
is_first: false,
tag: CopyDataType::Memory,
id: call_id,
```

```
addr: dest_offset + i,
...
);
}
```

CODECOPY

```
// CODECOPY(dest_offset, offset, length)
// 현재 컨트랙트의 바이트코드를 메모리로 복사

for i in 0..length {
// Read from Bytecode
lookup_copy_table(
tag: CopyDataType::Bytecode,
id: code_hash_rlc,
addr: offset + i,
src_addr_end: code_size,
...
);
// Write to Memory
lookup_copy_table(
tag: CopyDataType::Memory,
id: call_id,
addr: dest_offset + i,
```

```
...
);
}
```

EXTCODECOPY

```
// EXTCODECOPY(address, dest_offset, offset, length)
// 외부 컨트랙트의 바이트코드를 메모리로 복사

let external_address = stack.pop();
let external_code_hash = get_code_hash(external_address);

for i in 0..length {
// Read from external Bytecode
lookup_copy_table(
tag: CopyDataType::Bytecode,
id: external_code_hash_rlc,
addr: offset + i,
...
);
// Write to Memory
lookup_copy_table(
tag: CopyDataType::Memory,
```

```
id: call_id,
addr: dest_offset + i,
...
);
}
```

RETURNDATACOPY

```
// RETURNDATACOPY(dest_offset, offset, length)
// 이전 호출의 반환 데이터를 메모리로 복사

for i in 0..length {
// Read from Memory (caller's return data)
lookup_copy_table(
tag: CopyDataType::Memory,
id: last_callee_id,
addr: return_data_offset + offset + i,
...
);
// Write to Memory (current call)
lookup_copy_table(
tag: CopyDataType::Memory,
id: call_id,
addr: dest_offset + i,
```

```
...
);
}
```

LOG0-LOG4

```
// LOG2(offset, length, topic1, topic2)
// 메모리에서 로그로 복사

for i in 0..length {
// Read from Memory
lookup_copy_table(
tag: CopyDataType::Memory,
id: call_id,
addr: offset + i,
...
);
// Write to TxLog
lookup_copy_table(
tag: CopyDataType::TxLog,
id: tx_id,
addr: log_address(tx_id, log_id, Data, i),
...
```

```
);

}
```

Copy Circuit과의 연동

Copy Circuit은 Copy Table에 데이터를 할당하고 검증합니다.

데이터 할당

파일: `zkevm-circuits/src/table.rs` (line 1804)

```
impl CopyTable {

pub  fn  assignments<F: Field>(

copy_event: &CopyEvent,

challenges: Challenges<Value<F>>,

bytecode_map: Option<&BTreeMap<Word, bool>>,

) -> Vec<(CopyDataType, CopyTableRow<F>, CopyCircuitRow<F>)> {

// 1. RLC 누적 값 계산

// 2. Read/Write 쌍 생성

// 3. 각 바이트에 대해 행 생성

// 4. 패딩 처리

// 5. RW 카운터 업데이트

}

}
```

CopyEvent 구조체

파일: `bus-mapping/src/circuit_input_builder.rs`

```rust
pub struct CopyEvent {
pub src_type: CopyDataType,
pub dst_type: CopyDataType,
pub src_id: NumberOrHash,
pub dst_id: NumberOrHash,
pub src_addr: u64,
pub src_addr_end: u64,
pub dst_addr: u64,
pub log_id: Option<u64>,
pub rw_counter_start: usize,
pub copy_bytes: CopyBytes,
pub access_list: Vec<AccessListItem>,
}
```

제약 조건

Copy Circuit은 다음을 검증합니다:

1. **소스-목적지 일치**: Read 값과 Write 값이 동일
2. **주소 연속성**: 주소가 순차적으로 증가
3. **바이트 수 감소**: `real_bytes_left` 가 올바르게 감소
4. **RW 카운터 증가**: `rw_counter` 가 올바르게 증가

5. **패딩 처리**: `src_addr >= src_addr_end` 일 때 값이 0

성능 최적화

1. Binary Number Chip

`tag` 컬럼은 Binary Number Chip을 사용하여 효율적으로 인코딩:

- 여러 비트로 분해하여 저장
- 제약 조건 간소화

2. SecondPhase 컬럼

RLC를 사용하는 컬럼들은 SecondPhase에 할당:

- `id` : Bytecode Hash RLC
- `rlc_acc` : RLC 누적 값

3. 워드 단위 처리

메모리 복사는 32바이트 워드 단위로 그룹화하여 RW 카운터 최적화

예시: 전체 복사 이벤트

Solidity 코드:

```
function copyData(bytes calldata data) public {
bytes memory mem = new bytes(data.length);
// CALLDATACOPY 발생
assembly {
```

```
calldatacopy(add(mem, 32), 4, calldatasize())

}

}
```

Copy Table (CallData: `0x12345678`):

```
is_first | tag | id | addr | src_addr_end | real_bytes_left | rw_counter

---------|------------|----| -----|--------------|-----------------|------------

1 | TxCalldata | 1 | 0 | 4 | 4 | 100

0 | Memory | 1 | 32 | 36 | 4 | 101

0 | TxCalldata | 1 | 1 | 4 | 3 | 102

0 | Memory | 1 | 33 | 36 | 3 | 103

0 | TxCalldata | 1 | 2 | 4 | 2 | 104

0 | Memory | 1 | 34 | 36 | 2 | 105

0 | TxCalldata | 1 | 3 | 4 | 1 | 106

0 | Memory | 1 | 35 | 36 | 1 | 107
```

디버깅 팁

Copy Event 덤프

```
// 복사 이벤트 출력

for copy_event in block.copy_events.iter() {

println!("Copy: {:?} -> {:?}, length={}, rw_counter={}",
```

```
copy_event.src_type, copy_event.dst_type,

copy_event.copy_length(), copy_event.rw_counter_start);

}
```

일반적인 오류

1. **소스-목적지 불일치**: Read 값과 Write 값이 다름
2. **주소 불연속**: 주소가 순차적으로 증가하지 않음
3. **RW 카운터 오류**: rw_counter가 올바르게 증가하지 않음
4. **패딩 오류**: 소스 버퍼 끝 이후 값이 0이 아님

관련 문서

- Copy Circuit - Copy Table을 생성하는 회로
- EVM Circuit - Copy Table을 조회하는 회로
- RW Table - Read-Write 연산 테이블
- RLC 기법 - RLC 인코딩 상세

참고 자료

- **코드**: `zkevm-circuits/src/table.rs` (line 1728)
- **Copy Circuit**: `zkevm-circuits/src/copy_circuit.rs`
- **Bus-Mapping**: `bus-mapping/src/circuit_input_builder.rs`
- **EIP-5656**: https://eips.ethereum.org/EIPS/eip-5656 (MCOPY)